

EX NO : 12

ANN

DATE : 02-09-26

AIM:

To construct a 3-layer Artificial Neural Network (ANN) using TensorFlow/Keras for classifying the IRIS dataset, and to analyze performance differences upon increasing the number of hidden layers in the architecture.

OBJECTIVES

- **Part A:** Implement and train a basic 3-layer ANN on standardized Iris dataset features to establish a baseline classification accuracy.
- **Part B:** Modify the neural network by adding multiple hidden layers to evaluate changes in model complexity, convergence rate, and overall testing performance.

CODE:

```
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 1. Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Build 3-layer ANN (1 input specification + 1 hidden + 1 output)
model_3layer = Sequential(
    [Dense(8, activation="relu", input_shape=(4,)), Dense(3, activation="softmax")]
)

# 5. Compile model
model_3layer.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

# 6. Train model
model_3layer.fit(X_train, y_train, epochs=50, batch_size=8, verbose=1)

# 7. Evaluate
loss_a, acc_a = model_3layer.evaluate(X_test, y_test, verbose=0)
print(f"\nPart A - Test Loss: {loss_a:.4f} | Test Accuracy: {acc_a:.4f}")
```

```

# Predict sample
pred = model_3layer.predict(X_test[:1])
print(f"Actual: {y_test[0]}, Predicted: {pred.argmax()}")

# Build multi-layer ANN with 3 hidden layers
model_deep = Sequential([
    Dense(16, activation="relu", input_shape=(4,)), # Hidden Layer 1
    Dense(16, activation="relu"), # Hidden Layer 2
    Dense(8, activation="relu"), # Hidden Layer 3
    Dense(3, activation="softmax"), # Output Layer
])

# Compile model
model_deep.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

# Train model
model_deep.fit(X_train, y_train, epochs=50, batch_size=8, verbose=1)

# Evaluate deep model
loss_b, acc_b = model_deep.evaluate(X_test, y_test, verbose=0)
print(f"\nPart B - Test Loss: {loss_b:.4f} | Test Accuracy: {acc_b:.4f}")

```

OUTPUT:

```

Epoch 1/10
15/15 ————— 1s 5ms/step - accuracy: 0.4583 - loss: 1.1233
Epoch 2/10
15/15 ————— 0s 3ms/step - accuracy: 0.4583 - loss: 1.0263
Epoch 3/10
15/15 ————— 0s 3ms/step - accuracy: 0.5000 - loss: 0.9480
Epoch 4/10
15/15 ————— 0s 4ms/step - accuracy: 0.5417 - loss: 0.8758
Epoch 5/10
15/15 ————— 0s 3ms/step - accuracy: 0.6167 - loss: 0.8109
Epoch 6/10
15/15 ————— 0s 3ms/step - accuracy: 0.6833 - loss: 0.7559
Epoch 7/10
15/15 ————— 0s 3ms/step - accuracy: 0.7250 - loss: 0.7089
Epoch 8/10
15/15 ————— 0s 3ms/step - accuracy: 0.7583 - loss: 0.6691
Epoch 9/10
15/15 ————— 0s 3ms/step - accuracy: 0.7667 - loss: 0.6330
Epoch 10/10
15/15 ————— 0s 3ms/step - accuracy: 0.7667 - loss: 0.6026

Part A - Test Loss: 0.5760 | Test Accuracy: 0.8000

```

Part A - Test Loss: 0.5760 | Test Accuracy: 0.8000

1/1  0s 143ms/step

Actual: 1, Predicted: 2

Epoch 1/10

15/15  1s 4ms/step - accuracy: 0.2000 - loss: 1.1112

Epoch 2/10

15/15  0s 4ms/step - accuracy: 0.3083 - loss: 1.0732

Epoch 3/10

15/15  0s 4ms/step - accuracy: 0.3667 - loss: 1.0313

Epoch 4/10

15/15  0s 4ms/step - accuracy: 0.5500 - loss: 0.9616

Epoch 5/10

15/15  0s 4ms/step - accuracy: 0.6000 - loss: 0.8840

Epoch 6/10

15/15  0s 3ms/step - accuracy: 0.6500 - loss: 0.8285

Epoch 7/10

15/15  0s 3ms/step - accuracy: 0.6500 - loss: 0.7901

Epoch 8/10

15/15  0s 3ms/step - accuracy: 0.6583 - loss: 0.7589

Epoch 9/10

15/15  0s 3ms/step - accuracy: 0.6583 - loss: 0.7252

Epoch 10/10

15/15  0s 3ms/step - accuracy: 0.6583 - loss: 0.6866

Part B - Test Loss: 0.6240 | Test Accuracy: 0.7000